

课程笔记

CS224R 课程笔记: Tutorial Session

Review of Q-Learning

Codex

2026-04-13



视频作者/频道: Stanford Online

发布日期: 讲次日期 2025-04-18; YouTube 上传 2025-12-08

视频时长: 50:39

视频链接: <https://www.youtube.com/watch?v=07MQNMcxhZU>

目录

1 从 MDP 到 Q 函数: Q-Learning 在解什么问题	2
1.1 Value、Q-value 与 Advantage 的关系	2
1.2 本章小结	3
2 Tabular 场景中的 Bellman 递推与 Fitted Q Iteration	3
2.1 Bellman 最优方程	3
2.2 把 Bellman 方程变成迭代算法	3
2.3 为什么这一步重要	4
2.4 本章小结	4
3 从 tabular 到 parametric: 为什么需要神经网络 Q 函数	4
3.1 连续动作与离散动作的不同 Q 结构	5
3.2 本章小结	5
4 TD、Monte Carlo 与 N-Step: 偏差和方差的交换	5
4.1 从 Monte Carlo 到 TD	5
4.2 偏差与方差	6
4.3 本章小结	6
5 实践细节: target network、梯度稳定性、replay buffer 与过估计	7
5.1 为什么需要 semi-gradient 和 target network	7
5.2 TD 梯度为什么容易炸	7
5.3 Replay Buffer 的角色	8
5.4 过估计偏差从哪里来	8
5.5 本章小结	9
6 算法走查: DQN 与 Actor-Critic	9
6.1 DQN whiteboard	9
6.2 SAC whiteboard 与 actor-critic 视角	10
6.3 本章小结	10
7 总结与延伸	10

1 从 MDP 到 Q 函数: Q-Learning 在解什么问题

这一讲是助教习题课，目标不是重新发明 Q-learning，而是把前面课堂上的记号和更新规则真正串成一个可执行的算法视角。整个讲义从最基本的 MDP 出发，再逐步走到 fitted Q iteration、参数化 Q-learning 以及 DQN / SAC whiteboard。

MDP Formulation

An MDP is defined by:

- Set of states s_t
- Set of actions a_t
- Transition function $p(s_{t+1} | s_t, a_t)$
- Reward Function $r(s_t, a_t)$
- Start state s_1
- Discount Factor γ
- Horizon T

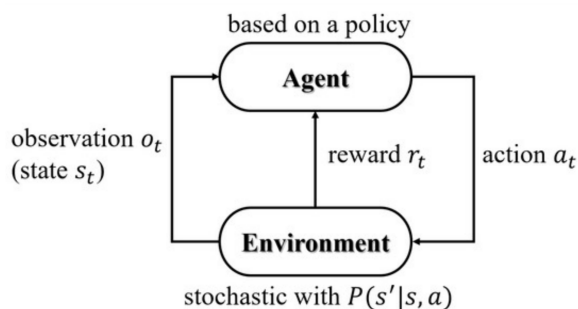


图 1: MDP 的基本组成: 状态、动作、转移、奖励、起点、折扣因子与 horizon。

在 MDP 中，我们关心的是一条策略 π 在长期回报上的表现。记号层面最重要的几个对象是：

- s_t : 状态；
- a_t : 动作；
- $p(s_{t+1} | s_t, a_t)$: 转移函数；
- $r(s_t, a_t)$: 奖励；
- γ : 折扣因子；
- T : 轨迹长度或有效决策视野。

Q-learning 之所以有吸引力，是因为它尝试直接学习

$$Q^\pi(s, a),$$

也就是“在状态 s 下先执行动作 a ，之后继续按某策略走，未来总回报的期望”。

1.1 Value、Q-value 与 Advantage 的关系

本讲也顺手回顾了另外两个对象：

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot|s)}[Q^\pi(s, a)], \quad A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

符号解释如下：

- $V^\pi(s)$ 只看状态，衡量“从这里开始总体有多好”；
- $Q^\pi(s, a)$ 把第一步动作也固定下来，因此更细；
- $A^\pi(s, a)$ 表示某个动作相对当前策略平均水平好多少。

这三个量后来会分别服务于 value-based、Q-based 与 actor-critic 方法。

1.2 本章小结

这一部分的核心是统一语言：Q-learning 不是在学一个抽象分数，而是在学“某状态下某动作的长期价值”。它与 value function、advantage function 一起构成后续所有 TD 方法的基础。

2 Tabular 场景中的 Bellman 递推与 Fitted Q Iteration

2.1 Bellman 最优方程

在已知转移模型的 tabular setting 里，最优 Q 函数满足 Bellman 最优方程：

$$Q^*(s_t, a_t) = \sum_{s_{t+1}} p(s_{t+1} | s_t, a_t) \left(r(s_t, a_t) + \gamma \max_{a_{t+1}} Q^*(s_{t+1}, a_{t+1}) \right).$$

这里每一项的含义是：

- $p(s_{t+1} | s_t, a_t)$ ：当前状态动作对会把你带到哪些下一状态；
- $r(s_t, a_t)$ ：立刻拿到的 reward；
- $\gamma \max_{a_{t+1}} Q^*(s_{t+1}, a_{t+1})$ ：从下一步开始能拿到的最优未来价值。

2.2 把 Bellman 方程变成迭代算法

直接求解固定点不一定方便，因此可以把它写成 value iteration / fitted Q iteration 的形式：

$$Q_{k+1}^*(s_t, a_t) = \sum_{s_{t+1}} p(s_{t+1} | s_t, a_t) \left(r(s_t, a_t) + \gamma \max_{a_{t+1}} Q_k^*(s_{t+1}, a_{t+1}) \right).$$

Fitted Q Iteration with the MDP

Discount = 0.9

$$Q_{k+1}^*(s, a) = \sum_{s_{t+1}} p(s_{t+1} | s_t, a_t) \left(r(s_t, a_t) + \gamma \max_{a_t} Q_k^*(s_{t+1}, a_{t+1}) \right)$$

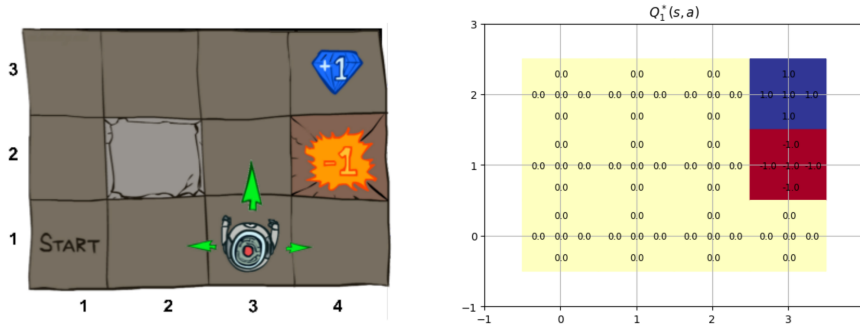


图 2: Fitted Q Iteration 的直观图示：从 terminal reward 开始，价值在网格中逐步向外传播。

这张图很好地说明了 FQI 的核心机制。初始时，除了终止状态的奖励之外，其余位置的 Q 值全为 0；每做一次迭代，终止状态附近的价值就向外扩散一层，最终整个状态空间都获得稳定的价值估计。

2.3 为什么这一步重要

很多人第一次看 Q-learning 会只记住更新公式，却忽略了它其实继承动态规划。理解 FQI 的传播过程很重要，因为后面的 TD、target network、double Q 等技巧，本质上都是围绕“如何更稳定地逼近这个 Bellman 固定点”展开的。

2.4 本章小结

Fitted Q Iteration 给出了 Q-learning 的最干净版本：已知模型、离散状态动作、从终止奖励向外传播价值。后续神经网络版只是把这个思路搬到无法枚举的大空间里。

3 从 tabular 到 parametric：为什么需要神经网络 Q 函数

一旦状态空间或动作空间变大，表格法很快不可用。存储和更新每个 (s, a) 对的成本都会爆炸，因此需要参数化的 value function。

Limitations of Tabular Environment

Moving towards parametric value functions

- A tabular value function intractable as state/action space grows!
- Learning a parametric value function can also allow for generalization
 - Related states have similar q-values (smoothness)
- Can use regression to learn value functions with a neural network

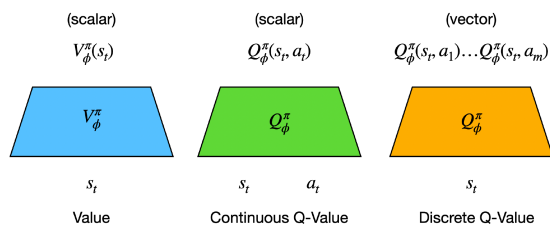


图 3: 从 tabular 到 parametric value functions: 状态相近时, 神经网络可以借助平滑性做泛化。

这一步带来两个变化。第一, 我们不再显式存储每个状态动作对的值, 而是用函数逼近器输出它们。第二, 泛化开始出现: 相似状态的值不再彼此独立, 而是通过共享参数互相影响。

3.1 连续动作与离散动作的不同 Q 结构

本讲特别提醒, 参数化 Q 函数的形状还取决于动作空间:

- 离散动作时, 网络可以一次性输出所有动作的 Q 值;
- 连续动作时, 通常要把动作也作为输入, 输出一个标量 $Q(s, a)$ 。

这也是为什么 DQN 适合离散动作, 而很多连续控制方法会自然走向 actor-critic: 因为在连续动作里直接对 $Q(s, a)$ 做 $\arg \max$ 往往不现实。

3.2 本章小结

参数化 Q-learning 的意义不只是节省内存, 更是用函数逼近获得泛化能力。但一旦用了神经网络, 也就把逼近误差、训练不稳定和过估计问题一起带进了进来。

4 TD、Monte Carlo 与 N-Step: 偏差和方差的交换

4.1 从 Monte Carlo 到 TD

Monte Carlo 的直觉是最直接的: 沿一整条轨迹把未来奖励全算完, 再回头回归。TD 则更激进, 它在还没看到整条轨迹结束之前, 就用当前 Q 估计去 bootstrap 下一步价值。

最常见的一步 TD 目标可以写成:

$$y_t = r_t + \gamma \max_{a'} Q_{\text{target}}(s_{t+1}, a').$$

这里:

- r_t 是当前即时奖励；
- Q_{target} 提供下一步的 bootstrap 目标；
- $\max_{a'} V_{a'}$ 表示选择下一状态下最优动作的估计价值。

4.2 偏差与方差

讲者用非常直观的方式总结了 TD、Monte Carlo 与 N-step 之间的差异。

Bias vs Variance Tradeoff

TD, MC, N-Step Return

Method	Bootstrap Depth	Bias	Variance
TD	1 Steps	High	Low
N-Step	N Steps	Medium	Medium
Monte Carlo	Full Episode	Low	High

图 4: TD、N-step 与 Monte Carlo 的 bias-variance trade-off。

可以把它概括为：

- TD: bootstrap 深度短，偏差高，但方差低；
- Monte Carlo: 看完整条轨迹，偏差低，但方差高；
- N-step: 在两者之间折中。

这不是谁绝对更好，而是你愿意接受多少噪声、又愿意忍受多少 bootstrap bias 的问题。语言模型长链 rollouts、机器人长时程控制、离线数据稀缺等情境，都会改变这个折中的最优点。

4.3 本章小结

TD 与 Monte Carlo 的差别，不只是更新公式不同，而是估计哲学不同。一个更相信未来估计值，一个更相信完整真实回报；N-step 则提供了连续可调的中间谱系。

5 实践细节：target network、梯度稳定性、replay buffer 与过估计

5.1 为什么需要 semi-gradient 和 target network

当 Q 网络同时出现在更新目标两边时，如果梯度穿透到目标分支，训练会变得非常不稳定。于是就有了 semi-gradient 和 target network：目标值由一个冻结或缓慢更新的网络给出，回归网络只对左边预测项求梯度。

5.2 TD 梯度为什么容易炸

Practical Implementation Details

Dealing with TD Gradients

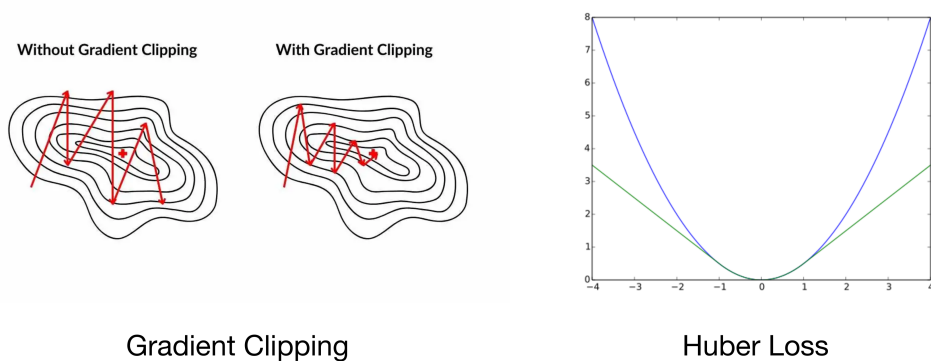


图 5: TD 训练中的两个常见稳定化技巧：gradient clipping 与 Huber loss。

TD 目标里的 bootstrap 误差会层层传递，因此梯度往往尖锐、不稳定。为此常见的两个技巧是：

- gradient clipping：限制异常大的梯度步长；
- Huber loss：在误差大时用线性增长代替二次增长，减少离群点破坏。

5.3 Replay Buffer 的角色

Practical Implementation Details

Replay Buffers

- Replay Buffers are **memory structures** that store past experiences encountered by the agent, allowing the **experiences to be reused** instead of discarded after a single update
- They **break temporal correlations** between consecutive training samples and prevents recency bias
- **Improves sample efficiency** in Q-learning algorithms, allowing an agent to reuse its experience effectively

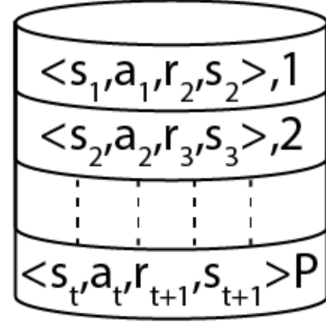


图 6: Replay buffer 让旧经验可以反复采样，既提升样本效率，也打破时间相关性。

replay buffer 的意义至少有两个。其一，数据可以重复利用，不必每次更新都重新采样环境；其二，随机采样打破了连续样本之间的强时间相关性，使监督回归更像 i.i.d. 训练。对于 Q-learning 这样的 off-policy 方法，这一点尤其关键。

5.4 过估计偏差从哪里来

Q-Learning Overestimation

Q-Learning is typically formulated as follows:

$$Q(s, a) \leftarrow r(s, a) + \gamma \max_{\hat{a}} Q(s', \hat{a})$$

When parameterized by a function approximator like a NN, there is some inherent noise in the q-value estimates due to modeling errors

$$Q^{\text{approx}}(s', \hat{a}) = Q^{\text{target}}(s', \hat{a}) + Y_{s'}^{\hat{a}}$$

Due to the noise on the RHS ($Y_{s'}^{\hat{a}}$), there results in error on the LHS, expressed as:

$$Z_s := \gamma \left(\max_{\hat{a}} Q^{\text{approx}}(s', \hat{a}) - \max_{\hat{a}} Q^{\text{target}}(s', \hat{a}) \right)$$

Even with zero mean ($Y_{s'}^{\hat{a}}$), Z_s may have positive mean:

$$\mathbb{E}[Y_{s'}^{\hat{a}}] = 0 \forall \hat{a} \implies \mathbb{E}[Z_s] > 0 \text{ (often)}$$

Issues in Using Function Approximation for Reinforcement Learning

图 7: Q-learning overestimation: 即使估计噪声是零均值，经过 max 运算后也容易产生正偏。

过估计的根源不神秘：如果每个动作价值估计都带噪声，那么取 $\max$ 会系统性偏向噪声偏大的动作。因此即使单个估计误差均值为 0，最大值的期望依然可能为正。Double Q-learning / double critic 正是用“选择”和“评估”分离来缓解这个偏差。

5.5 本章小结

Q-learning 真正难的地方不只在 Bellman 公式，而在函数逼近后的稳定训练。target network、gradient clipping、Huber loss、replay buffer 和 double Q 都是在处理这个现实问题。

6 算法走查：DQN 与 Actor-Critic

6.1 DQN whiteboard

Q-Learning : DQN Whiteboard

Algorithm 1: DQN

Input: Replay buffer $\mathcal{D}$, Q-network Q_θ , target network $Q_{\theta'}$ with $\theta' \leftarrow \theta$, discount factor γ , batch size N , learning rate α , target update frequency C , exploration schedule ϵ

for each interaction step $t = 1, 2, \dots$ **do**

 Observe state s_t

 Choose action a_t using ϵ -greedy:

$$a_t = \begin{cases} \text{random action,} & \text{with probability } \epsilon, \\ \arg \max_a Q_\theta(s_t, a), & \text{otherwise} \end{cases}$$

 Execute a_t , observe (r_t, s_{t+1}) , and store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$

 // Sample a mini-batch

 Sample $\{(s_i, a_i, r_i, s_{i+1})\}_{i=1}^N \sim \mathcal{D}$

 // Compute targets

for $i = 1, \dots, N$ **do**

$y_i \leftarrow r_i + \gamma \max_{a'} Q_{\theta'}(s_{i+1}, a')$

 // Q-network update

$\theta \leftarrow \theta - \alpha \nabla_\theta \frac{1}{N} \sum_{i=1}^N [Q_\theta(s_i, a_i) - y_i]^2$

 // Periodic target update

if $t \bmod C = 0$ **then**

$\theta' \leftarrow \theta$

图 8: DQN whiteboard：收集转移、从 replay buffer 采样、构造 target、更新 Q 网络、周期性同步 target network。

DQN 的执行流程可以简化为：

1. 用 ϵ -greedy 与环境交互，收集 (s_t, a_t, r_t, s_{t+1}) ；
2. 把数据放入 replay buffer；
3. 从 buffer 采样 mini-batch；
4. 用 target network 构造 y_i ；
5. 最小化 $(Q_\theta(s_i, a_i) - y_i)^2$ ；
6. 周期性更新 target network。

它是离散动作 Q-learning 最典型的工程化实现。

6.2 SAC whiteboard 与 actor-critic 视角

Actor Critic: SAC Whiteboard

Algorithm 1: Soft Actor-Critic (no entropy term)

Input: Replay buffer $\mathcal{D}$, critic nets $Q_{\theta_1}, Q_{\theta_2}$, actor net π_{ϕ} , target critics $\theta'_j \leftarrow \theta_j$, discount γ , smoothing τ , batch size N

for each interaction step $t = 1, 2, \dots$ **do**
 Observe state s_t
 Sample action $a_t \sim \pi_{\phi}(\cdot | s_t)$
 Execute a_t , observe (r_t, s_{t+1}) and store in $\mathcal{D}$
 // Sample a mini-batch
 Sample $\{(s_i, a_i, r_i, s_{i+1})\}_{i=1}^N \sim \mathcal{D}$
 // Compute targets (no entropy term)
 for $i = 1, \dots, N$ **do**
 $a'_{i+1} \leftarrow \pi_{\phi}(s_{i+1})$;
 $y_i \leftarrow r_i + \gamma \min_{j=1,2} Q_{\theta'_j}(s_{i+1}, a'_{i+1})$
 // Critic update
 for $j = 1, 2$ **do**
 $\theta_j \leftarrow \theta_j - \lambda_Q \nabla_{\theta_j} \frac{1}{N} \sum_i [Q_{\theta_j}(s_i, a_i) - y_i]^2$
 // Actor update (maximize Q only)
 $\phi \leftarrow \phi + \lambda_{\pi} \nabla_{\phi} \frac{1}{N} \sum_i Q_{\theta_1}(s_i, \pi_{\phi}(s_i))$
 // Target networks soft-update
 for $j = 1, 2$ **do**
 $\theta'_j \leftarrow \tau \theta_j + (1 - \tau) \theta'_j$

图 9: SAC whiteboard: critic 负责评估动作价值, actor 负责输出连续动作分布, 两者通过双重 critic 与软更新耦合。

SAC 把思路推广到连续动作控制。与 DQN 最大的差别在于: 不再直接对动作做枚举式 $\arg \max$, 而是显式学习 actor 来产出动作, 再由一个或多个 critic 评估这些动作。这样, 连续控制问题就从“在 $Q(s, a)$ 上做全局搜索”改成了“用策略网络学会去 critic 认为高价值的区域采样”。

把本讲串起来的主线

tabular FQI、TD learning、DQN、SAC 并不是彼此割裂的方法。它们都在围绕同一个问题工作: 如何稳定逼近 Bellman 固定点, 并据此改进策略。

6.3 本章小结

白板算法部分把所有前面的概念落到了真正可运行的训练 loop 上。DQN 展示了离散动作下的标准配方, SAC 则展示了连续动作下 actor-critic 的自然扩展。

7 总结与延伸

这节 tutorial 的价值, 在于把很多课堂上分散的概念重新接上线。Q-learning 不是一句更新式, 而是一整套从 MDP 建模、Bellman 递推、函数逼近、目标稳定化到算法工程实现的连续链条。

把全讲压缩成一句话就是:

Q-learning 通过 bootstrap 近似长期价值, 但一旦进入函数逼近世界, 稳定性和估计偏差就成为第一公民。

因此, 真正要带走的不是某个公式, 而是以下结构:

- 先理解 V, Q, A 的角色分工;

- 再理解 Bellman 递推如何转成 FQI / TD 更新；
- 再理解为什么神经网络会带来梯度不稳与过估计；
- 最后理解 DQN 和 SAC 如何把这些问题工程化解决。

如果继续往后学，这些内容会自然通向 offline RL、actor-critic 变体、distributional RL 和现代大模型后训练中的 value estimation 问题。Q-learning 看似老，但它的核心张力今天仍然没有过时。